Chord Recognition from a Live Audio Stream

John Montesa
Dept. of Computer Science and Engineering
University of Nevada, Reno
CS 631
Reno, United States

***Abstract*— When playing with other musicians, especially during improvisation, it is a challenge to "catch" on to what others are playing while also focusing on playing the instrument at the same time. Visually identifying chords by watching other musicians can be distracting and listening to figure out what key or chords other musicians are playing by ear is an extremely difficult skill to learn. Having an easier way to figure out what chord is being played in real time will allow for easier music transcription, efficient practicing, and effortless improvisation.**

Keywords—Guitar, Improvisation, Music, Machine Learning, Chords, Convolutional Neural Network, Spectrogram

I. Introduction

Chords are the foundational structure for music where combinations of notes create tonalities that define genres, emotions, and musical harmony. Being able to recognize chords in real-time scenarios like improvisation sessions is a challenging, but important task for musicians to learn. Experienced players are able to recognize chords by ear, however it requires years of experience and training, which is still susceptible to error in noisy settings. Identifying chords with sight is challenging as well as it can require clear vision of other players and it can distract a player if they are focusing on playing music. To bridge this gap of complexity for musicians, a real-time chord recognition script is developed to aid musicians in improvisation and to motivate players to learn these difficult skills that can improve themselves as musicians.

The main functionality of the script should be able to accurately detect chord changes within songs and guitars. It will utilize a machine learning algorithm to train from a dataset of audio files. After training, then the user is able to run the script on their device and recognize various guitar chords. The script will be constructed in Python utilizing various libraries to access input devices to stream audio, convert audio files to spectrograms, and utilize machine learning.

II. Related Works

There are two applications that take similar approaches: Shazam and Chordify. Shazam is an music recognition system that compares a snippet of live-streamed audio to its database of audio files. Similar to this approach, it tries to identify common frequencies to find a match. Shazam pioneered the use of spectrograms and spectrogram peaks [1] as an industry standard for music recognition. With their approach, they combine multiple spectrogram frequency peaks to create a fingerprint for each song. These fingerprints are used to match songs together.

Chordify is another software that recognizes chords through uploaded audio samples, however this is not in real time. Chordify also has premium features locked behind a paywall, lowering access to many musicians.

Other academic approaches include a classification model by Huang and Zhang [2] to utilize a two stage process using a hybrid hidden Markov model and artificial neural network to classify music. By grouping audio together by broad categorization, they were able to accurately classify music into genres offline. Because this is offline, however, it is not able to do real-time recognition, compared to the proposed script.

III. Motivation

Musicians, especially in live or improvisation settings, tend to find it difficult to interpret what others are playing, or simply lack the technical skill to identify chords in real-time. This task, especially for beginners, can be extremely challenging and can demotivate players to learn these skills. It requires extensive training and years of experience to develop the technical skills for ear-training and chord recognition. With a script that can recognize chords from a live-audio stream, the gap between technical skill and chord recognition can be bridged. The script serves as a valuable learning tool for musicians. Having an "instant" response to a chord played can catalyze ear-training by providing immediate feedback to how a chord sounds and is played.

The script can also be used to learn songs by ear, furthermore enhancing the skills of a musician to play by ear. Motivating musicians to learn music by themselves instead of other sources will strengthen their musicianship and creativity, allowing them to focus on their musical expression and experiences. Ultimately, the goal is to create an accessible and intuitive tool that musicians can utilize to practice, play together, and engage players with their instruments.

IV. Approach

A. Setup

The main premise behind the solution is to train a machine learning algorithm to recognize a chord's spectrogram image using a convolutional neural network (CNN). The user would then be able to train the algorithm with the dataset. After training, the user would run the script and the script should recognize the guitar chords a user plays or, in the case of music, the script would recognize guitar chords in a song. The

script should be able to recognize chords in real-time to conform to musical scenarios such as live improvisation.

B. Dataset

The datasets used include a chord labeling and an audio file. This means the script takes a supervised learning approach. Folders are nested within the dataset folders as the label, for example, a folder named "C" will hold all C chord audio files. Two datasets were utilized: "GUITAR CHORDS V3" by Fabiana Vinci [1] and a custom dataset. The first dataset includes 8 chords (A minor, B flat major, B diminished, C major, D minor, E minor, F major, and G major), each with multiple recordings utilizing different techniques, recording devices, guitars, and volumes. In total, each folder had 40 audio files.

A custom dataset was created to include 3 more chords (B major, C minor, and D major), and also included more chord variations (Major 7 chords, different positioned chords, etc.) to include more tonality recognition. In total, there were 220 more audio files created and aggregated with the "GUITAR CHORDS V3."

C. Spectrogram

In order to train the machine learning algorithm, the CNN needs structured input data to read. To solve this issue, each audio file was converted into a spectrogram. A spectrogram is a visual representation of sound as a spectrum of frequencies over time chart. Figure 1 shows an example of an audio file (G major chord strummed once) used in the dataset as a spectrogram.

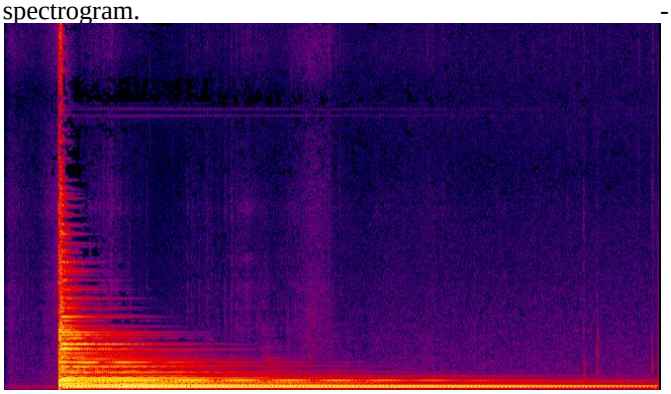

Fig . 1. Spectrogram of a G major chord strummed

Utilizing this transformation, the CNN can analyze spatial patterns in the spectrograms to classify chords accurately. Because chords are structured and retain the same harmonies or frequencies, it makes it possible to recognize these when taking in a real-time audio stream.

D. Convolution Neural Network

The code for the script was created in Python using TensorFlow to train the CNN, which is used for classifying chords and processing spectrogram images to identify chord characteristics and features. There are three convolutional layers followed by max pooling. Before output, the strategy also does global average pooling. The script does 50 passes of training to make sure the training is adequate. An adam optimizer is also used to create adaptive learning rates and improve efficiency. The algorithm follows a 80/20 split where 80% of the data is used for training and 20% is used for validation which helps generalize and prevent overfitting.

V. Evaluation

A. Analysis of Solution

In its current state, the script can recognize 11 different guitar chords. The terminal interface consists of small volume bar to show microphone detection, a chord detected, and the chords' confidence score. Figure 2 shows the current interface when the script is run.

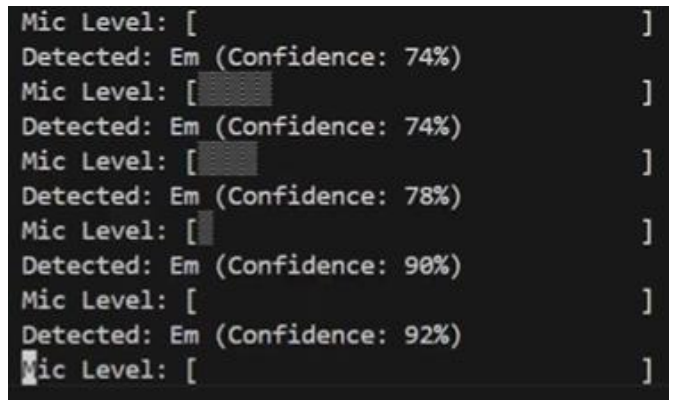

Fig . 2. Implemented terminal interface.

A couple tests were conducted to test the usability and accuracy of the script. First, a chord detection accuracy test was conducted. Next, a song chord detection test was conducted to test how accurate the script was detecting guitar chords in music.

B. Chord Detection Accuracy Test

To conduct this test, 3 random videos were found to teach a user how to play an E minor, F major, and G major. The user was then supposed to play the chord exactly how the video taught it. This test was conducted with an acoustic guitar and no effects. Figure 3 illustrates the results of the test with a G major chord.

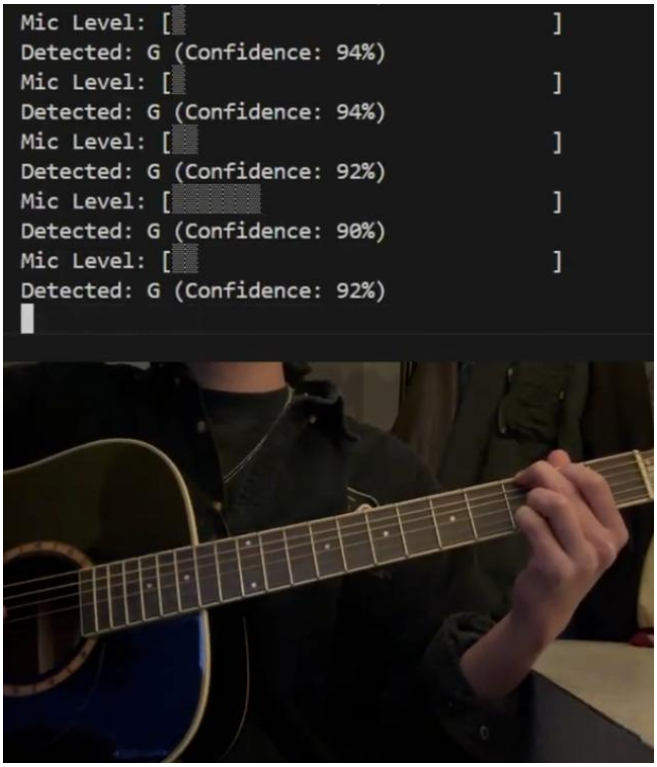

Fig. 3. Chord detection accuracy test with a G major chord and an acoustic guitar.

The results of this test showed accurate detection for the chords used for the test. It was able to effectively detect the chord while staying confident in its findings. This test represents one use case of the script, which is being able to identify guitar chords in real-time, which is essential for improvisation purposes and ear training.

C. Music Chord Detection Test

To conduct this test, the input device for the script was switched to the computer's stereo mix, meaning that any audio played from the computer will be picked up by the script. For this test, Radiohead's "Creep" was played through the computer to see if the script can detect any chords within its chord progression. Figure 4 shows the test running through one chord progression, with the music video being played in real-time.

Fig. 4. Music chord detection test, with the script running on the right side and "Creep" by Radiohead on the left

The results of this test showed that the algorithm picked up a G major, B major, C major, and C minor during a full chord progression of the song. Additionally, the script was tested with an electric guitar with effects (tremolo, chorus, and reverb) to see how the script does with different guitars and sounds. Figure 5 shows the script picking up an electric guitar while playing the same chords picked up by the script when it was "listening" to "Creep" by Radiohead. when it was "listening" to "Creep" by Radiohead.

Fig. 5. Music chord detection test, with the script running on the right side and "Creep" by Radiohead on the left

Playing the "learned" chords of G, B, C, and C minor over the song fit perfectly, effectively allowing a user to learn a song without having to look it up online. This is particularly useful in cases where songs are thought up on the spot during improvisation and in cases where songs do not yet have tutorials online and a user wants to learn how to play it.

D. Limitations

The script is limited to the recognition of guitar chords, as it is only trained to recognize the frequencies and harmonies of guitar chords. There are 5 major chords (B, Bb, C, G, D, and F), 5 minor chords (Cm, Em, Dm, and Am), and B diminished that can be recognized by the script. Chord

variations like maj7 or 5 chords can be recognized, however only as its base chord (e.g., C5 being recognized as a C).

There is also some latency before detecting chord changes. It takes approximately a second for the script to correctly recognize the chord change. It takes even more time to recognize changes from chords with the same base. Changing from C major to C minor, because of similar frequencies and harmonies, takes considerably longer to recognize than a C major to a G major.

Finally, the script is limited to the terminal of Visual Studio Code. Due to time constraints, a full user interface was unable to be developed. Chord recognition, volume levels, and all interfacing is done through the terminal and through .

VI. Conclusion and Future Work

In conclusion, the implemented script was able to accurately guess what guitar chord was being played if the algorithm was trained to detect that chord. There is a slight delay when a chord change is detected, however this can be due to a variety of reasons such as hardware issues (microphone) or if a chord change is similar to the previous chord (C to C minor), which would require more training data to be able to detect the minor differences in frequency. This is one definite improvement that will be happening in the future.

Currently some of the basic guitar chords can be recognized (major and minor chords). In the future, chord variations like 7th, 9th, sus chords, augmented, and chords for other instruments should be able to be detected. Being able to recognize these chords requires an extensive training set to identify the minuscule changes in frequency. However, allowing the script to identify each chord variation properly opens up many more use cases, especially for advanced players that use more chord variations.

The script is also limited to the terminal of Visual Studio Code. Incorporating the script into a user interface for both mobile and web applications will allow for increased accessibility. This would entail full-stack development; however, it would be worthwhile creating a full application for this script as it has not yet been made.

Another future idea was encoding the chords detected to other functions. This is useful in performance scenarios where we can encode the chord to a lighting system, for example. As a chord changes, a performer can use lighting and color to create more audience engagement and provoke emotion through color and music.

References

[1] A. Wang, ‘The Shazam music recognition service’, *Commun. ACM*, vol. 49, no. 8, pp. 44–48, Aug. 2006.

[2] W. Huang and Y. Zhang, ‘Application of Hidden Markov Chain and Artificial Neural Networks in Music Recognition and Classification’, in Proceedings of 2020 6th International Conference on Computing and Data Engineering, Sanya, China, 2020, pp. 49–53

[3] F. Vinci, "GUITAR CHORDS V3," 2023. [Online]. Available: https://www.kaggle.com/datasets/fabianavinci/guitar-chords-v3. Accessed: May 7, 2025.